

Traffic Violations Insight System

What it does, how to run it, and a full code walkthrough

Traffic Violations Insight System

Exploratory Data Analysis - Data Cleaning - Interactive Streamlit Dashboard

What this project does

Takes a raw, messy traffic-violations export (inconsistent state codes, misspelled vehicle makes, duplicate rows, free-text violation descriptions) and turns it into something usable: a cleaning pipeline that fixes those problems, a written EDA report with real charts, and an interactive dashboard where you can filter ~70,000 stops by violation type, state, vehicle type, gender, race, or arrest type and see the summary numbers and charts update live. It's built for anyone who wants to explore patterns in who gets stopped, for what, and in what kind of vehicle - a city transportation office, a researcher, or just someone curious about the data.

Cleans a raw traffic violations export, engineers a few derived features, and serves the result through an interactive Streamlit dashboard plus a static EDA report with saved charts.

A note on the dataset

This project is modeled on the full Montgomery County-style traffic stops dataset, which includes stop date/time, agency, location, and lat/long. The `raw_traffic.csv` used here is a **trimmed export that doesn't include those columns** - it has description, boolean incident flags, state/jurisdiction codes, vehicle info, and driver demographics, but no date, time, or location data. That rules out anything involving hotspots, time-of-day, or weekday/month trends - those questions just aren't answerable with this file. `clean_data.py` is written so it would pick date/time/geo columns up automatically if a fuller export were dropped in later, but nothing here currently depends on them.

Project Objectives

- Clean and de-duplicate ~70,000 traffic violation records
- Standardize state codes, vehicle makes, and boolean incident flags
- Flag out-of-range vehicle years and invalid state/jurisdiction codes
- Categorize free-text violation descriptions into a fixed set of groups
- Derive an accident-severity indicator from the incident flags
- Explore common violations, demographics, and vehicle involvement in a written EDA report with saved charts
- Serve everything through a filterable Streamlit dashboard

What's Here

File	Purpose
<code>clean_data.py</code>	Reads <code>raw_traffic.csv</code> , cleans and enriches it, writes <code>cleaned_traffic.parquet</code>
<code>eda.py</code>	Reads the cleaned data, saves charts to <code>eda_charts/</code> , writes <code>EDA_REPORT.md</code>

<code>app.py</code>	Streamlit dashboard over the cleaned data
<code>EDA_REPORT.md</code>	Generated by <code>eda.py</code> - written findings with embedded charts
<code>eda_charts/</code>	Generated by <code>eda.py</code> - the PNGs referenced in the report
<code>cleaned_traffic.parquet</code>	Generated by <code>clean_data.py</code> - not committed, regenerate locally

Data cleaning (`clean_data.py`)

- Drops full-row duplicates (99 found in the raw export)
- Maps Yes/No/Y/N flags to real booleans, leaving unrecognized values as missing rather than assuming "No"
- Uppercases state/jurisdiction codes and nulls out known junk values (`XX`, `US`) - Canadian province codes are left alone since those are legitimate
- Blanks out vehicle years outside 1960-2025 (typos like `1013` or placeholders like `0/9999`)
- Collapses common make abbreviations (`TOYT` -> `TOYOTA`, `HOND` -> `HONDA`, etc.) so one manufacturer doesn't get split across rows in the charts
- Buckets free-text violation descriptions into groups (Speeding, Red Light / Signal, Registration / Plate, License / Suspended, Seatbelt, Stop Sign, DUI / Alcohol, Other)
- Derives an ordinal `accident_severity` column from the injury/damage/ contributed-to-accident flags
- Converts low-cardinality text columns to `category` dtype and vehicle year to a nullable int, to keep the parquet file small

EDA (`eda.py`)

Answers the questions the data can support: most common violations, vehicle types most involved, accident-contribution rate by vehicle type, and how violation type breaks down by race and gender. Descriptive numbers and the generated charts live in `EDA_REPORT.md` once you run it.

Dashboard (`app.py`)

- Filters: violation group, driver's state, vehicle type, gender, race, arrest type
- Summary tiles: total violations, accident-contributed count, seatbelt violations, personal injury cases, plus the most frequently cited make/model in the current selection
- Charts: violation categories, charge codes, race x gender, vehicle makes (treemap), vehicle type breakdown, arrest type outcomes
- Download the filtered slice as CSV

Setup

```
pip install -r requirements.txt
```

Running It

```
# 1. Clean the raw data (creates cleaned_traffic.parquet)
python clean_data.py

# 2. Generate the EDA report and charts (creates EDA_REPORT.md, eda_charts/)
python eda.py

# 3. Launch the dashboard
streamlit run app.py
```

The dashboard opens at <http://localhost:8501>.

How to Use the Dashboard

1. **Pick your filters in the sidebar** - Violation Group, Driver's State, Vehicle Type, Gender, Race, and Arrest Type. Leaving a filter empty means "don't filter on this" - it opens with Speeding/Registration-Plate/ License-Suspended and State=MD pre-selected so you're not staring at every row on first load.
2. **Read the summary tiles** at the top - total violations, how many contributed to an accident, seatbelt violations, personal-injury cases, and (below the tiles) the single most-cited make/model in your current selection.
3. **Expand the chart sections** - "Violation Categories & Patterns" is open by default; "Demographics", "Vehicle Information", and "Enforcement & Arrest" are collapsed, click to expand. Every chart updates to match whatever filters are currently active.
4. **Download the filtered slice** - the CSV button exports exactly the rows matching your current filters, not the whole dataset.

Try it: clear every filter except Vehicle Type = **10 - Transit Bus**, and compare the accident-contribution rate you see against the same rate for **02 - Automobile** - this is exactly the pattern flagged in the Recommendations section below.

Recommendations

Based on [EDA_REPORT.md](#):

- **Transit buses and school buses show a disproportionately high accident-contribution rate** relative to their share of stops. Worth a closer look even though the raw counts are small - a targeted inspection or training program for those fleets could be cheap to run and high value.
- **"Other" is the largest violation-group bucket** across every demographic group, which mostly means the keyword rules in `categorize_violation()` don't cover enough ground yet. Expanding that function (or moving to a proper text-classification pass) would sharpen every other chart in the dashboard.
- **To actually answer the hotspot/time-of-day questions this project was scoped for, the dataset needs a fuller export** with stop date, time, and location. The cleaning pipeline is already written to pick those up if they show up later.

Troubleshooting

Problem	Solution
<code>streamlit</code> command not found	Use <code>python -m streamlit run app.py</code> instead
No module named 'plotly' (or seaborn/pyarrow)	Run <code>pip install -r requirements.txt</code>

Error loading parquet file	Run <code>python clean_data.py</code> first
<code>eda.py</code> can't find the parquet file	Same - run <code>clean_data.py</code> before <code>eda.py</code>

Traffic Violations Insight System -- How to Run It & Code Walkthrough

This document has two parts:

1. **How to run the project**, start to finish, on a clean machine.
2. **A line-by-line explanation of every script** -- `clean_data.py`, `eda.py`, and `app.py` -- so you can follow exactly what each line does and why.

For the "what does the spec ask for vs. what does this data actually have" context, see [README.md](#). This document is purely about the code.

Part 1 -- How to Run the Project

1. Prerequisites

- Python 3.9+ (anything recent works)
- `raw_traffic.csv` sitting in the project root (already there if you cloned this repo)

2. Install dependencies

From the project folder:

```
pip install -r requirements.txt
```

This installs `pandas`, `numpy`, `streamlit`, `plotly`, `pyarrow`, `seaborn`, and `matplotlib`. `pyarrow` is what actually lets pandas read/write the `.parquet` file format used between scripts -- without it, both `clean_data.py` and `app.py` will fail at the read/write step.

If you're using a virtual environment (recommended so this doesn't collide with other projects' package versions):

```
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate   # macOS/Linux
pip install -r requirements.txt
```

3. Run the pipeline, in this exact order

The three scripts depend on each other's output, so order matters:

```
python clean_data.py      # step 1: raw_traffic.csv -> cleaned_traffic.parquet
python eda.py             # step 2: cleaned_traffic.parquet -> EDA_REPORT.md + eda_charts/
streamlit run app.py      # step 3: cleaned_traffic.parquet -> interactive dashboard
```

- `clean_data.py` reads `raw_traffic.csv` and writes `cleaned_traffic.parquet`. Nothing else works until this has run at least once. Re-run it any time `raw_traffic.csv` changes -- it always regenerates the parquet from

scratch.

- `eda.py` reads `cleaned_traffic.parquet` and writes `EDA_REPORT.md` plus a handful of PNG charts into `eda_charts/`. This step is independent of step 3 -- you can skip it if you only want the dashboard.
- `streamlit run app.py` launches the dashboard at `http://localhost:8501`. If the browser doesn't open automatically, visit that URL manually.

4. Using the dashboard

The sidebar has six filters -- Violation Group, Driver's State, Vehicle Type, Gender, Race, Arrest Type. Leaving a filter empty means "don't filter on this dimension at all" (not "show nothing"). The four metric tiles at the top, the vehicle caption underneath them, and every chart below all update together whenever a filter changes. "Reset all filters" clears every one of them back to empty in one click.

The "Download Filtered Data (CSV)" button exports exactly the rows currently matching your filters, not the full dataset.

5. If something goes wrong

Symptom	Fix
<code>streamlit</code> command not found	Run <code>python -m streamlit run app.py</code> instead
<code>ModuleNotFoundError: No module named 'X'</code>	<code>pip install -r requirements.txt</code> -- you're probably missing a package or in the wrong virtual environment
<code>FileNotFoundError: cleaned_traffic.parquet</code> (from <code>app.py</code> or <code>eda.py</code>)	Run <code>python clean_data.py</code> first
Dashboard loads but charts are empty	Check the sidebar filters aren't all narrowed down to zero matching rows

Part 2 -- Code Walkthrough

`clean_data.py`

This script takes the messy raw CSV and turns it into a cleaned, type-optimized parquet file. It's built as a pipeline of small functions, each responsible for exactly one kind of cleaning, chained together at the bottom in `clean_traffic_data()`.

Lines 1-11 -- module docstring

```
"""Cleans and enriches the raw Montgomery County-style traffic violations export.
...
"""
```

Explains what the script does and, importantly, flags up front that this particular CSV is missing whole categories of columns (date/time, location) that the full public dataset normally has. That context matters for understanding why several functions later on have `if column not in df.columns: return df` guards -- they're not being overly defensive for no reason, they're anticipating columns that genuinely aren't there.

Lines 13-16 -- imports

```
import sys
import numpy as np
import pandas as pd
```

`sys` is only used once, in `load_raw_data()`, to exit cleanly if the input file is missing. `numpy` is used for `np.nan` and `np.select`. `pandas` does all the actual data work.

Lines 18-19 -- file path constants

```
RAW_FILE = "raw_traffic.csv"
OUTPUT_FILE = "cleaned_traffic.parquet"
```

Named once here instead of hard-coded as string literals scattered through the file, so changing the input/output filename later only means touching one line.

Lines 21-31 -- `BOOLEAN_COLUMNS`

```
BOOLEAN_COLUMNS = [
    "belts",
    "personal_injury",
    "property_damage",
    "commercial_license",
    "commercial_vehicle",
    "contributed_to_accident",
]
```

These are the six Yes/No columns that actually exist in `raw_traffic.csv` (after column names get lowercased with underscores -- more on that in `normalize_column_names`). The comment above the list explicitly notes that the full dataset also has Fatal, HAZMAT, Alcohol, and Work Zone flags, but since those columns aren't in this export, they're left out of the list rather than pretending to clean columns that don't exist.

Lines 33-39 -- state code constants

```
STATE_COLUMNS = ["state", "driver_state", "dl_state"]
INVALID_STATE_CODES = {"XX", "US"}
```

Three separate columns in the data carry two-letter jurisdiction codes: where the stop happened, where the driver lives, and which state issued their license. `INVALID_STATE_CODES` is a set (not a list) purely for O(1) membership checks later via `.isin()` -- with only two values the performance difference is meaningless here, but it's the right data structure for a "is this value in this fixed collection" check. `"XX"` and `"US"` were found by inspecting the actual unique values in the data -- `"XX"` is a generic unknown/placeholder code, `"US"` looks like someone typed the country instead of a state. Real Canadian province codes (`ON`, `QC`, `MB`, etc.) also show up in the data but are legitimate -- they're not in this set, so they survive cleaning untouched.

Lines 41-58 -- `MAKE_ALIASES`

```
MAKE_ALIASES = {
    "TOYT": "TOYOTA",
    "HOND": "HONDA",
    "CHEV": "CHEVROLET",
    "CHEVY": "CHEVROLET",
    "NISS": "NISSAN",
}
```

```

"HYUN": "HYUNDAI",
"MERZ": "MERCEDES-BENZ",
"MERCEDES": "MERCEDES-BENZ",
"VOLK": "VOLKSWAGEN",
"VW": "VOLKSWAGEN",
"ACUR": "ACURA",
"INFI": "INFINITI",
"MIT": "MITSUBISHI",
}

```

A lookup dictionary for `Series.replace()` later. Officers apparently abbreviate manufacturer names inconsistently when filing citations -- `"TOYT"` and `"TOYOTA"` both show up in the raw data as if they were different manufacturers. Without this mapping, the "top vehicle makes" chart in the dashboard would split Toyota's count across two rows. Two aliases can point at the same target (`CHEV` and `CHEVY` both become `CHEVROLET`), which is exactly what a dict allows and a simpler `if/elif` chain would make tedious.

Lines 60-61 -- vehicle year bounds

```

MIN_VEHICLE_YEAR = 1960
MAX_VEHICLE_YEAR = 2025

```

Straight from the project spec ("remove impossible years (<1960 or >2025)"). Named constants instead of magic numbers inline in `clean_vehicle_year()`.

Lines 63-69 -- GENERIC_MODEL_CODES

```

GENERIC_MODEL_CODES = {
    "4S", "2S", "4D", "4DR", "2D", "2DR", "4 DOOR", "2 DOOR",
    "SUV", "VN", "VAN", "SW", "TK", "TRUCK", "SU",
}

```

Some rows have a body-style shorthand (`"4S"` = 4-door sedan, `"TK"` = truck) sitting in the `Model` column instead of an actual model name -- almost certainly because the officer didn't know or didn't note the specific model. These get treated as missing data rather than real models later in `clean_vehicle_model()`; otherwise `"4S"` would show up as the single most common "model" in the whole dataset, ahead of every real car model.

Lines 72-79 -- load_raw_data()

```

def load_raw_data(path=RAW_FILE):
    """Read the raw CSV export. Bails out with a clear message if it's missing."""
    try:
        df = pd.read_csv(path, low_memory=False)
    except FileNotFoundError:
        sys.exit(f"Could not find {path} - make sure it's in the project folder.")
    print(f"Loaded {len(df):,} rows from {path}")
    return df

```

`path=RAW_FILE` as a default argument means you can call `load_raw_data("some_other_file.csv")` for testing without touching the function body. `low_memory=False` tells pandas to read the whole file before guessing column dtypes, rather than guessing chunk-by-chunk -- this dataset mixes types often enough (e.g. the `Year` column has both clean floats and garbage strings in some exports) that chunk-based guessing throws `DtypeWarning`. The `try/except` catches specifically `FileNotFoundError` (not a bare `except`, which would also swallow unrelated bugs) and `sys.exit(message)` prints the message and stops the script with a non-zero exit code, which is friendlier than pandas' own raw traceback for someone running this from the command line.

Lines 82-90 -- `normalize_column_names()`

```
def normalize_column_names(df):
    """lowercase_with_underscores instead of 'Personal.Injury' / 'Driver City'."""
    df.columns = (
        df.columns.str.strip()
        .str.lower()
        .str.replace(".", "_", regex=False)
        .str.replace(r"\s+", "_", regex=True)
    )
    return df
```

The raw CSV's headers are inconsistent -- some use dots (`Personal.Injury`), some use spaces (`Driver City`), all use mixed case. This chains four string operations on the whole `Index` of column names at once (pandas lets you call `.str` methods on an `Index` just like a `Series`): strip whitespace, lowercase everything, turn literal dots into underscores (`regex=False` because `.` is a regex metacharacter and we want a literal dot here, not "any character"), then collapse any run of whitespace into a single underscore (`regex=True`, `\s+` matches one-or-more whitespace characters). After this, `"Personal.Injury"` becomes `personal_injury` and `"Driver City"` becomes `driver_city` -- which is why every other function in this file refers to columns using the snake_case names.

Lines 93-102 -- `drop_duplicate_rows()`

```
def drop_duplicate_rows(df):
    """Full-row duplicates showed up in the raw export - most likely re-exported
    or double-scanned records rather than two separate stops that happen to
    match on every single field."""
    before = len(df)
    df = df.drop_duplicates().reset_index(drop=True)
    removed = before - len(df)
    if removed:
        print(f"Removed {removed} duplicate rows")
    return df
```

`df.drop_duplicates()` with no arguments compares every column across every row and drops rows that are 100% identical to an earlier one -- the odds of two genuinely different stops matching on every single field (description, make, model, color, charge code, race, gender, city...) are effectively zero, so a full match really is a duplicate record, not a coincidence. `.reset_index(drop=True)` rennumbers the `DataFrame`'s row index from 0 upward again after rows have been dropped (otherwise the index would have gaps where duplicates used to be) -- `drop=True` throws the old index away instead of keeping it as a new column. The before/after row-count comparison and print statement exist purely so running the script tells you how many rows were actually removed, rather than cleaning silently.

Lines 105-126 -- `clean_boolean_columns()`

```
def clean_boolean_columns(df):
    """Map the Yes/No/Y/N text columns to real booleans.

    ...
    """
    yes_no_map = {
        "yes": True, "y": True,
        "no": False, "n": False,
    }
    for col in BOOLEAN_COLUMNS:
        if col not in df.columns:
            print(f>Note: expected boolean column '{col}' not found, skipping")
            continue
        mapped = df[col].astype(str).str.strip().str.lower().map(yes_no_map)
```

```

unmapped = df[col].notna() & mapped.isna()
if unmapped.any():
    bad_values = df.loc[unmapped, col].unique()
    print(f"'{col}': {unmapped.sum()} rows had unrecognized values {list(bad_values)[:5]}")
df[col] = mapped
return df

```

`yes_no_map` is deliberately lowercase-keyed, because the actual comparison happens after `.str.lower()` -- so this one dict correctly matches "Yes", "YES", "yes", and "Y"/"y" all at once without needing every casing variant spelled out. The loop walks each of the six known boolean columns: `if col not in df.columns` guards against a column being absent (defensive, in case a future export drops one) and just logs a note rather than crashing. `df[col].astype(str).str.strip().str.lower()` forces every value to a string first (in case pandas read a mixed-type column oddly), trims stray whitespace, lowercases it, then `.map(yes_no_map)` looks each cleaned value up in the dictionary -- any value not in the dict becomes `NaN`. The `unmapped` line is the important part: it flags rows where the original value was *not* null (`df[col].notna()`) but the mapping still produced `NaN` (`mapped.isna()`) -- meaning there was some unrecognized spelling in the data that wasn't null and wasn't a clean Yes/No either. If any exist, it prints up to 5 example bad values so you can see what they were, instead of silently guessing. The important design decision is in the docstring: unmapped/blank values stay `NaN` (missing) rather than being defaulted to `False` -- a blank field means "we don't know", not "confirmed no".

Lines 129-137 -- `clean_state_codes()`

```

def clean_state_codes(df):
    """Uppercase state/jurisdiction codes and null out the known junk values."""
    for col in STATE_COLUMNS:
        if col not in df.columns:
            continue
        df[col] = df[col].astype(str).str.strip().str.upper()
        df[col] = df[col].replace("NAN", np.nan)
        df.loc[df[col].isin(INVALID_STATE_CODES), col] = np.nan
    return df

```

Loops over all three state-like columns. `.astype(str)` on a column that already has real `NaN` values turns those into the literal text "nan" -- that's why the very next line, `.replace("NAN", np.nan)`, exists: it puts genuine missing values back to `NaN` after the uppercasing step turned "nan" into "NAN". Without that line, actual missing states would get silently misread as the literal string "NAN" everywhere downstream. The last line, `df.loc[df[col].isin(INVALID_STATE_CODES), col] = np.nan`, uses boolean indexing: `df[col].isin(INVALID_STATE_CODES)` produces a True/False Series (True wherever the value is "XX" or "US"), and `.loc[mask, col] = np.nan` sets only those matching rows' value in that column to `NaN`, leaving everything else -- including legitimate non-U.S. codes -- untouched.

Lines 140-152 -- `clean_vehicle_year()`

```

def clean_vehicle_year(df):
    """Vehicle model years outside a sane range ..."""
    if "year" not in df.columns:
        return df
    df["year"] = pd.to_numeric(df["year"], errors="coerce")
    out_of_range = ~df["year"].between(MIN_VEHICLE_YEAR, MAX_VEHICLE_YEAR)
    invalid_count = (out_of_range & df["year"].notna()).sum()
    if invalid_count:
        print(f"Blanked out {invalid_count} out-of-range vehicle years")
    df.loc[out_of_range, "year"] = np.nan
    return df

```

`pd.to_numeric(..., errors="coerce")` converts the column to a proper numeric dtype, turning anything that can't be parsed as a number into `NaN` instead of raising an error. `df["year"].between(MIN_VEHICLE_YEAR, MAX_VEHICLE_YEAR)` returns True for every value from 1960 to 2025 inclusive; the `~` in front negates it, so `out_of_range` is True wherever the year is *outside* that window (this also correctly marks existing `NaN` years as "out of range", since `NaN.between(...)` is False and its negation is True -- which is fine, since the next line only counts/blanks values that are *already numbers*). `invalid_count` combines `out_of_range` with `df["year"].notna()` specifically so the printed count only reflects years that were genuinely present-but-invalid (like 0, 9999, or 1013), not years that were already missing before this function ran. The last line blanks out every out-of-range value (including the pre-existing `NaN`s, harmlessly re-setting them to `NaN`).

Lines 155-162 -- `standardize_make()`

```
def standardize_make(df):
    """Collapse the obvious abbreviation duplicates ...
    """
    if "make" not in df.columns:
        return df
    df["make"] = df["make"].astype(str).str.strip().str.upper()
    df["make"] = df["make"].replace(MAKE_ALIASES)
    return df
```

Uppercases and trims the make column first, so the alias dictionary only needs to list uppercase keys ("`TOYT`", not also "`toyt`" and "`Toyt`"). `.replace(MAKE_ALIASES)` swaps any value matching a dictionary key for its corresponding value -- values not in the dictionary (e.g. "`TOYOTA`" itself, which is already correct) pass through unchanged.

Lines 165-174 -- `clean_vehicle_model()`

```
def clean_vehicle_model(df):
    """Null out generic body-style codes ...
    """
    if "model" not in df.columns:
        return df
    df["model"] = df["model"].astype(str).str.strip().str.upper()
    df.loc[df["model"].isin(GENERIC_MODEL_CODES), "model"] = np.nan
    return df
```

Same pattern as `clean_state_codes()`: uppercase/trim first, then use `.isin()` plus `.loc[...] = np.nan` to blank out only the rows whose model value is one of the known generic body-style codes ("`4S`", "`TK`", "`VAN`", etc.), leaving real model names untouched.

Lines 177-201 -- `categorize_violation()`

```
def categorize_violation(description):
    """Bucket the free-text violation description into a handful of groups
    used throughout the dashboard. Order matters here - ...
    """
    if pd.isna(description):
        return "Unknown"

    text = str(description).lower()

    if "speed" in text or "exceeding" in text:
        return "Speeding"
    if "red light" in text or "traffic signal" in text:
        return "Red Light / Signal"
    if ("registration" in text or "expired" in text) and "plate" in text:
```

```

        return "Registration / Plate"
    if "license" in text or "suspended" in text:
        return "License / Suspended"
    if "seatbelt" in text or "belt" in text or "restrained" in text:
        return "Seatbelt"
    if "stop sign" in text:
        return "Stop Sign"
    if "alcohol" in text or "influence" in text:
        return "DUI / Alcohol"
    return "Other"

```

This function is applied to one description string at a time (via `.apply()` later). `pd.isna(description)` handles a missing description up front. `text = str(description).lower()` normalizes case once so every keyword check below can be a plain lowercase substring match. Each `if` returns immediately on the first matching category -- the order is deliberate: for example, `"registration"/"expired"` is checked with an explicit `and` `"plate" in text` so a description needs *both* an expired/registration keyword *and* the word "plate" to land in that bucket, avoiding a false match on, say, "expired license" (which should fall through to "License / Suspended" instead). The parentheses around `("registration" in text or "expired" in text)` matter for order of operations -- without them, Python's `and` binds tighter than `or`, so `"registration" in text or "expired" in text and "plate" in text` would actually mean `"registration" in text or ("expired" in text and "plate" in text)`, letting any bare mention of "registration" match regardless of "plate". Anything matching none of the keyword groups falls through to the final `return "Other"`.

Lines 204-229 -- `add_accident_severity()`

```

def add_accident_severity(df):
    """Roll the three accident-related flags into one ordinal severity label,
    worst outcome wins. ...
    """
    injury = df.get("personal_injury")
    damage = df.get("property_damage")
    contributed = df.get("contributed_to_accident")

    if injury is None or damage is None or contributed is None:
        return df

    all_missing = injury.isna() & damage.isna() & contributed.isna()
    severity = np.select(
        [
            injury.fillna(False),
            damage.fillna(False),
            contributed.fillna(False),
        ],
        ["Personal Injury", "Property Damage", "Accident (No Injury/Damage Noted)"],
        default="No Accident",
    )
    severity = pd.Series(severity, index=df.index)
    severity[all_missing] = "Unknown"
    df["accident_severity"] = severity
    return df

```

`df.get("personal_injury")` returns the column if it exists or plain `None` if it doesn't -- different from `df["personal_injury"]`, which would raise a `KeyError` on a missing column. The `if ... is None` check bails out entirely if any of the three source columns is missing, since there's no sensible severity label without all three. `all_missing` flags rows where none of the three flags had any real data at all (all three were `NaN`). `np.select(conditions, choices, default=...)` walks the list of boolean conditions in order and returns the *first* matching choice for each row -- this is what makes the label "ordinal" (worst outcome wins): if `personal_injury` is True, the row gets `"Personal Injury"` regardless of what the other two flags say; only if that's False does it check `property_damage`, and so on; if none are True, it falls back to `"No Accident"`. Each condition uses

`.fillna(False)` specifically so a missing individual flag doesn't accidentally satisfy `np.select`'s truthiness check (a `NaN` is falsy for this purpose, but being explicit is safer than relying on that). `np.select` returns a plain numpy array, not a pandas Series, so `pd.Series(severity, index=df.index)` wraps it back up with the DataFrame's original row index -- necessary so the next line, `severity[all_missing] = "Unknown"`, can correctly align the `all_missing` boolean mask (which does have an index) against `severity` by row.

Lines 232-245 -- `engineer_features()`

```
def engineer_features(df):
    """Everything derived rather than read straight from the source columns."""
    if "description" in df.columns:
        df["violation_group"] = df["description"].apply(categorize_violation)

    df = add_accident_severity(df)

    if "make" in df.columns:
        make_counts = df["make"].value_counts()
        df["make_violation_count"] = df["make"].map(make_counts)

    return df
```

This is the function that actually calls `categorize_violation()`: `.apply(categorize_violation)` runs that function once per row of the `description` column and stores the resulting category string in a new `violation_group` column. It then calls `add_accident_severity()` to attach that column too. The last block computes, for every distinct make, how many total violations involve that make (`df["make"].value_counts()` returns a Series indexed by make, valued by count), then `df["make"].map(make_counts)` looks up each row's own make in that Series and broadcasts the count back onto every row -- so every Honda row gets the total Honda count, every Toyota row gets the total Toyota count, etc., without a manual merge/join.

Lines 248-262 -- `optimize_dtypes()`

```
def optimize_dtypes(df):
    """Low-cardinality text columns are stored as category dtype ...
    """
    categorical_candidates = [
        "state", "driver_state", "dl_state", "race", "gender", "vehicletype",
        "violation_type", "arrest_type", "violation_group", "accident_severity",
    ]
    for col in categorical_candidates:
        if col in df.columns:
            df[col] = df[col].astype("category")

    if "year" in df.columns:
        df["year"] = df["year"].astype("Int32") # nullable int, keeps NaNs

    return df
```

Pandas' `category` dtype stores each distinct string value once internally and represents every row as a small integer code pointing at that value, rather than repeating the full string ~70,000 times -- a meaningful memory saving for columns like `state` or `gender` that only have a handful of distinct values across tens of thousands of rows. Only columns that actually exist get converted (`if col in df.columns`), so this list can safely include column names even if a future dataset variant is missing one. `"Int32"` (capital I) is pandas' *nullable* integer type -- plain numpy `int` columns can't hold `NaN` at all, which would force `year` back to `float64` (and lose the "this is a whole year number" intent) the moment any row is missing a year. `Int32` keeps missing years as a proper `<NA>` while storing everything else as an actual integer.

Lines 265-280 -- `clean_traffic_data()`

```
def clean_traffic_data():
    """Run the full pipeline and write out cleaned_traffic.parquet."""
    df = load_raw_data()
    df = normalize_column_names(df)
    df = drop_duplicate_rows(df)
    df = clean_boolean_columns(df)
    df = clean_state_codes(df)
    df = clean_vehicle_year(df)
    df = standardize_make(df)
    df = clean_vehicle_model(df)
    df = engineer_features(df)
    df = optimize_dtypes(df)

    df.to_parquet(OUTPUT_FILE, index=False, engine="pyarrow")
    print(f"\nSaved {len(df):,} cleaned rows -> {OUTPUT_FILE}")
    return df
```

The orchestrator -- every earlier function takes a DataFrame and returns a DataFrame, so this just threads `df` through all of them in the order that matters (e.g. duplicates are dropped before spending time cleaning columns on rows that are about to be discarded; `standardize_make` runs before `engineer_features` so the per-make violation counts are computed on already-standardized make names, not on the raw abbreviations).

`df.to_parquet(OUTPUT_FILE, index=False, engine="pyarrow")` writes the final result to disk -- `index=False` skips writing pandas' row index as a column in the file (it's just 0, 1, 2, ... and not meaningful data), and `engine="pyarrow"` is explicit about which parquet library to use (pandas supports pyarrow or fastparquet; only pyarrow is in `requirements.txt`).

Lines 283-284 -- script entry point

```
if __name__ == "__main__":
    clean_traffic_data()
```

Standard Python idiom: `clean_traffic_data()` only runs automatically when this file is executed directly (python `clean_data.py`), not when it's imported by something else (nothing currently imports it, but this is cheap insurance for the future).

eda.py

Generates the static EDA deliverable: PNG charts saved to `eda_charts/`, and a written Markdown report (`EDA_REPORT.md`) that embeds those charts and narrates the numbers behind them.

Lines 1-14 -- module docstring

Explains how to run it, what it produces, and -- same honesty as `clean_data.py` -- states plainly that hotspot/time-of-day/weekday questions from the project spec aren't answered here because the underlying data doesn't have date/time/location columns.

Lines 16-20 -- imports

```
import os
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
```

`os` is used for `os.makedirs` and `os.path.join`. `matplotlib.pyplot` does the actual figure/axis drawing. `seaborn` is used narrowly -- just for `sns.set_theme()` (a nicer default look than raw matplotlib) and `sns.despine()` (removes the top/right border lines from each chart) -- everything else is plain matplotlib/pandas plotting.

Lines 22-24 -- file path constants

```
DATA_FILE = "cleaned_traffic.parquet"
CHART_DIR = "eda_charts"
REPORT_FILE = "EDA_REPORT.md"
```

Same "name it once" pattern as `clean_data.py`'s `RAW_FILE/OUTPUT_FILE`.

Lines 26-29 -- shared color palette

```
BLUE = "#2a78d6"
CATEGORICAL = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#4a3aa7"]
```

These exact hex values are duplicated in `app.py` on purpose (see that file's comment) -- a chart in the static report and the same kind of chart in the live dashboard use identical colors, so switching between the two doesn't feel like looking at a different project.

Line 31 -- seaborn theme

```
sns.set_theme(style="white", rc={"axes.grid": False})
```

Sets matplotlib's default style globally for every figure created after this line. `style="white"` gives a plain white background instead of seaborn's default gray-grid look; `rc={"axes.grid": False}` explicitly disables gridlines on top of that, since gridlines add visual noise on simple ranked bar charts where the bars themselves are the only thing that needs reading.

Lines 34-38 -- `load_cleaned_data()`

```
def load_cleaned_data():
    try:
        return pd.read_parquet(DATA_FILE)
    except FileNotFoundError:
        raise SystemExit(f"{DATA_FILE} not found - run clean_data.py first.")
```

Same pattern as `clean_data.py`'s `load_raw_data()`: catch the specific missing-file error and turn it into a clear one-line instruction instead of a pandas traceback. `raise SystemExit(message)` is equivalent to `sys.exit(message)` used in `clean_data.py` -- both print the message and exit with a non-zero code; this file just spells it slightly differently.

Lines 41-50 -- `save_horizontal_bar()`

```
def save_horizontal_bar(series, title, filename, color=BLUE):
    fig, ax = plt.subplots(figsize=(8, 5))
    series.sort_values().plot.barh(ax=ax, color=color)
    ax.set_title(title)
    ax.set_xlabel("Number of stops")
    ax.set_ylabel("")
    sns.despine(ax=ax, left=True)
    fig.tight_layout()
    fig.savefig(os.path.join(CHART_DIR, filename), dpi=150)
```

```
plt.close(fig)
```

A small reusable helper so the two ranked-bar charts (`chart_top_violations` and `chart_vehicle_types`) don't duplicate this same nine-line block twice. `plt.subplots(figsize=(8, 5))` creates one figure and one set of axes at a fixed 8x5 inch size, so every chart using this helper comes out the same size.

`series.sort_values().plot.barh(...)` sorts the input Series (e.g. violation-group counts) ascending and draws it as a horizontal bar chart -- matplotlib draws horizontal bars from the bottom of the axis upward, so sorting ascending puts the *largest* value at the *top* of the chart, which reads naturally as a ranked list. `ax.set_ylabel("")` blanks out the y-axis label because the category names themselves (shown as tick labels) already make it obvious what's being measured -- an extra axis label would be redundant. `sns.despine(ax=ax, left=True)` removes the top, right, and left border spines (the left spine is redundant once the bars themselves mark where zero is). `fig.tight_layout()` auto-adjusts spacing so labels don't get clipped or overlap the plot area. `fig.savefig(..., dpi=150)` writes the PNG at 150 dots-per-inch (sharp enough for the embedded markdown report without being an enormous file).

`plt.close(fig)` explicitly frees the figure from memory -- without this, matplotlib keeps every figure open until the process exits, which adds up if a script creates many charts in one run.

Lines 53-56 -- `chart_top_violations()`

```
def chart_top_violations(df):
    top = df["violation_group"].value_counts().head(8)
    save_horizontal_bar(top, "Most Common Violation Categories", "top_violations.png")
    return top
```

`.value_counts()` counts how many rows fall into each distinct `violation_group` value, already sorted descending by count; `.head(8)` keeps only the top 8. The function both draws the chart (via the helper above) *and* returns the counts, because `build_report()` later needs the actual numbers (e.g. "Speeding: 12,345 stops") to write into the Markdown text, not just the picture.

Lines 59-62 -- `chart_vehicle_types()`

Identical shape to `chart_top_violations()`, just counting `vehicletype` instead of `violation_group`.

Lines 65-82 -- `chart_race_by_violation()`

```
def chart_race_by_violation(df):
    top_groups = df["violation_group"].value_counts().head(5).index
    subset = df[df["violation_group"].isin(top_groups)]
    counts = pd.crosstab(subset["violation_group"], subset["race"])
    counts = counts.loc[top_groups]

    fig, ax = plt.subplots(figsize=(9, 5.5))
    counts.plot.bar(ax=ax, color=CATEGORICAL[: len(counts.columns)])
    ax.set_title("Top Violation Categories by Driver Race")
    ax.set_ylabel("Number of stops")
    ax.set_xlabel("")
    ax.legend(title="Race", bbox_to_anchor=(1.02, 1), loc="upper left")
    plt.xticks(rotation=20, ha="right")
    sns.despine(ax=ax)
    fig.tight_layout()
    fig.savefig(os.path.join(CHART_DIR, "race_by_violation.png"), dpi=150)
    plt.close(fig)
    return counts
```

This is the "demographics vs violation type" chart from the spec. First, `top_groups` grabs just the *names* of the five most common violation groups (`.index` on a `value_counts()` result gives you the category labels, not the counts) -- this keeps the chart to a readable number of groups instead of cramming in every category. `subset` filters the full

dataset down to only rows in one of those five groups. `pd.crosstab(a, b)` builds a cross-tabulation table: rows are the distinct values of the first argument (violation group), columns are the distinct values of the second (race), and each cell is a count of how many rows had that exact combination -- exactly the "do certain demographics correlate with specific violation types" question from the spec, as one table. `counts.loc[top_groups]` re-orders the crosstab's rows to match the original count-descending order (crosstab alphabetizes by default, which would otherwise scramble the ranking). `counts.plot.bar(...)` draws a *grouped* bar chart directly from that crosstab -- pandas automatically treats each column (race) as its own colored series and each row (violation group) as a cluster of bars along the x-axis. `color=CATEGORICAL[: len(counts.columns)]` slices the shared palette down to exactly as many colors as there are race categories, so colors are assigned in the same fixed order every time this chart is generated (never re-cycled based on which values happen to be present). `ax.legend(title="Race", bbox_to_anchor=(1.02, 1), loc="upper left")` moves the legend just outside the right edge of the plot area (`bbox_to_anchor` coordinates are in axes-fraction units, where `1.0` is the plot's right edge) so it doesn't overlap any bars. `plt.xticks(rotation=20, ha="right")` angles the x-axis category labels 20 degrees so longer names like "Registration / Plate" don't collide with their neighbors.

Lines 85-101 -- `chart_accident_rate_by_vehicle()`

```
def chart_accident_rate_by_vehicle(df):
    rate = (
        df.groupby("vehicletype", observed=True)["contributed_to_accident"]
        .mean()
        .sort_values(ascending=False)
        .head(8)
    ) * 100
    fig, ax = plt.subplots(figsize=(8, 5))
    rate.sort_values().plot.barh(ax=ax, color="#eb6834")
    ax.set_title("Accident-Contribution Rate by Vehicle Type")
    ax.set_xlabel("% of stops where the violation contributed to an accident")
    ax.set_ylabel("")
    sns.despine(ax=ax, left=True)
    fig.tight_layout()
    fig.savefig(os.path.join(CHART_DIR, "accident_rate_by_vehicle.png"), dpi=150)
    plt.close(fig)
    return rate
```

This chart answers a sharper question than a raw count would: not "which vehicle type gets stopped most" (that's `chart_vehicle_types`, and the answer is trivially "automobiles" since they're most of the traffic), but "which vehicle type's stops are most likely to involve an accident." `df.groupby("vehicletype", observed=True)["contributed_to_accident"].mean()` groups all rows by vehicle type, then averages the (already boolean) `contributed_to_accident` column within each group -- since `True/False` average out to a proportion (e.g. 0.25 means 25% of that vehicle type's stops involved an accident), this is a rate, not a count. `observed=True` matters because `vehicletype` is a *category* dtype column (set in `optimize_dtypes`); without it, `groupby` on a categorical column includes every category that *could* exist even if zero rows actually have it, producing empty/NaN groups in the output. `.sort_values(ascending=False)` then `.head(8)` keeps the 8 highest-rate vehicle types, and `* 100` converts the 0-to-1 proportion into a percentage for the axis label. This chart doesn't reuse `save_horizontal_bar()` because it needs a different x-axis label ("% of stops...") and a different bar color (orange, to visually distinguish it as "a rate" rather than "a raw count" like the other horizontal bars) -- otherwise it's the same shape.

Lines 104-114 -- `chart_gender_split()`

```
def chart_gender_split(df):
    counts = df["gender"].value_counts()
    fig, ax = plt.subplots(figsize=(5, 5))
    ax.bar(counts.index.astype(str), counts.values, color=CATEGORICAL[: len(counts)])
    ax.set_title("Stops by Gender")
```

```

ax.set_ylabel("Number of stops")
sns.despine(ax=ax)
fig.tight_layout()
fig.savefig(os.path.join(CHART_DIR, "gender_split.png"), dpi=150)
plt.close(fig)
return counts

```

A simple vertical bar chart (only 3 categories: M/F/U, so a small square figure and vertical bars both fit comfortably). `counts.index.astype(str)` converts the category labels to plain strings before handing them to matplotlib -- `gender` is a `category` dtype column, and some matplotlib versions handle categorical index objects awkwardly as bar-chart x-values, so this sidesteps that. `color=CATEGORICAL[: len(counts)]` again slices the shared palette to exactly the number of bars being drawn.

Lines 117-187 -- `build_report()`

```

def build_report(df, top_violations, top_vehicles, accident_rate, gender_counts):
    total = len(df)
    accident_total = int(df["contributed_to_accident"].sum())
    injury_total = int(df["personal_injury"].sum())
    damage_total = int(df["property_damage"].sum())
    make_model_counts = df.dropna(subset=["make", "model"]).groupby(["make", "model"]).size()
    top_make, top_model = make_model_counts.idxmax()

    lines = [ ... ]

    with open(REPORT_FILE, "w", encoding="utf-8") as f:
        f.write("\n".join(lines))
    print(f"Wrote {REPORT_FILE}")

```

Takes the already-computed chart data as arguments (rather than recalculating anything) so the numbers in the written report always match exactly what the charts show. `int(df["contributed_to_accident"].sum())` sums a boolean column -- `True` counts as 1, `False` as 0 -- giving a plain row count, wrapped in `int()` because pandas sums sometimes come back as a numpy integer type that formats slightly differently in an f-string. `make_model_counts = df.dropna(subset=["make", "model"]).groupby(["make", "model"]).size()` is worth calling out specifically: it groups by the *combination* of make and model together (dropping rows missing either first), so `.idxmax()` on the result returns the `(make, model)` pair that occurs together most often -- e.g. `("HONDA", "ACCORD")`. This is deliberately *not* two separate `df["make"].value_counts().idxmax()` and `df["model"].value_counts().idxmax()` calls, because the single most common make and the single most common model aren't guaranteed to belong to the same vehicle -- running those independently earlier in development actually produced the nonsensical "Toyota Accord" (Accord is a Honda model), which is why this line groups them jointly instead.

The `lines` list (omitted above for brevity, see the file itself) is just a list of Markdown-formatted strings -- headings, embedded image references like `![Top violations](eda_charts/top_violations.png)`, and narrative sentences built with f-strings that plug in the actual computed numbers (e.g. `f"{accident_total:,}"` formats a number with thousands separators). Building the report as a list of lines and joining with `"\n".join(lines)` at the end, rather than repeatedly appending to one long string, is just a cleaner way to lay out a multi-paragraph document in code -- each list entry reads as one line of the eventual file. The `with open(REPORT_FILE, "w", encoding="utf-8") as f:` block writes the whole joined string to disk in one call; `"w"` mode means it fully overwrites the file each time this runs (intentional -- the report is meant to be fully regenerated, not appended to), and `encoding="utf-8"` is explicit so special characters (the `x` in "Race x Gender Distribution", the `->/-` variants elsewhere) don't cause an encoding error on a system whose default text encoding isn't UTF-8.

Lines 190-201 -- `run_eda()`

```
def run_eda():
    os.makedirs(CHART_DIR, exist_ok=True)
    df = load_cleaned_data()

    top_violations = chart_top_violations(df)
    top_vehicles = chart_vehicle_types(df)
    chart_race_by_violation(df)
    accident_rate = chart_accident_rate_by_vehicle(df)
    gender_counts = chart_gender_split(df)

    build_report(df, top_violations, top_vehicles, accident_rate, gender_counts)
    print(f"Charts saved to {CHART_DIR}/")
```

The orchestrator for this file, same role as `clean_traffic_data()` in the other script. `os.makedirs(CHART_DIR, exist_ok=True)` creates the `eda_charts/` folder if it doesn't exist yet; `exist_ok=True` means it won't raise an error if the folder is already there from a previous run. Each chart function is called once, and the ones whose return value is needed later (violation counts, vehicle counts, gender counts) are captured into variables; `chart_race_by_violation(df)`'s return value isn't needed for the report text, so it's called without assigning the result anywhere.

Lines 204-205 -- entry point

```
if __name__ == "__main__":
    run_eda()
```

Same idiom as `clean_data.py`.

app.py

The Streamlit dashboard. Streamlit re-runs the *entire script top to bottom* every time a user interacts with a widget (changes a filter, clicks a button) -- that execution model shapes a lot of the structure here: state that needs to survive a rerun lives in `st.session_state`, and expensive work (loading the parquet file) is wrapped in `@st.cache_data` so it isn't redone on every single interaction.

Lines 1-7 -- module docstring

States how to run the file and what it expects to already exist (`cleaned_traffic.parquet`).

Lines 9-11 -- imports

```
import pandas as pd
import plotly.express as px
import streamlit as st
```

`streamlit as st` is the UI framework itself; `plotly.express as px` builds the interactive charts (hover tooltips, zoom, etc. come for free with Plotly, unlike the static matplotlib charts in `eda.py`); `pandas` handles the data filtering.

Lines 13-21 -- module-level constants

```
DATA_FILE = "cleaned_traffic.parquet"
CATEGORICAL_COLORS = ["#2a78d6", "#eb6834", "#1baf7a", "#eda100", "#e87ba4", "#4a3aa7"]
BLUE_SCALE = [[0, "#cde2fb"], [1, "#0d366b"]]
FILTER_KEYS = ["violation_group", "state", "arrest_type", "gender", "race", "vehicletype"]
```

`CATEGORICAL_COLORS` matches `eda.py`'s palette exactly (see that file's walkthrough). `BLUE_SCALE` is a two-stop *continuous* color scale (light blue at 0, dark blue at 1) for charts where color encodes a magnitude (like "how tall is this bar") rather than a category -- Plotly interpolates smoothly between the two stops for values in between. `FILTER_KEYS` lists the column names every sidebar filter is keyed on; it's used later by `reset_filters()` to know which widget keys to clear.

Lines 24-39 -- `configure_page()`

```
def configure_page():
    st.set_page_config(
        page_title="Traffic Violations Insight Dashboard",
        page_icon="",
        layout="wide",
        initial_sidebar_state="expanded",
    )
    st.markdown(
        """
        <style>
        .stMetric { background-color: #f0f2f6; border-radius: 10px; padding: 10px; }
        .block-container { padding-top: 1rem !important; }
        </style>
        """,
        unsafe_allow_html=True,
    )
```

`st.set_page_config(...)` must be the very first Streamlit command that runs in the whole script (Streamlit enforces this) -- it sets the browser tab title, the little emoji favicon, and tells Streamlit to use the full browser width (`layout="wide"`) with the sidebar already open on load. The `st.markdown(...)` call injects raw CSS through a `<style>` block: `unsafe_allow_html=True` is required because Streamlit escapes HTML in markdown by default (a security precaution against accidentally rendering user-supplied HTML) -- this override is safe here because the HTML is a fixed string written by the developer, not user input. The two CSS rules give the metric tiles a light gray rounded background and shave down Streamlit's default top padding so the page content starts closer to the top of the browser window.

Lines 42-54 -- `load_data()`

```
@st.cache_data
def load_data():
    """Load the cleaned parquet file. Raises so the caller can show a
    Streamlit-friendly error instead of a bare traceback."""
    df = pd.read_parquet(DATA_FILE)
    bool_cols = [
        "belts", "personal_injury", "property_damage",
        "commercial_license", "commercial_vehicle", "contributed_to_accident",
    ]
    for col in bool_cols:
        if col in df.columns:
            df[col] = df[col].astype(bool)
    return df
```

`@st.cache_data` is a decorator: Streamlit runs this function once, caches its return value, and on every subsequent script rerun (which happens on every user interaction) it returns the cached DataFrame instantly instead of re-reading the parquet file from disk each time. Without this decorator, every single filter click would re-read and re-process the whole ~70k-row file -- pointless, since the underlying data never changes during a session. The function itself just loads the parquet file and makes sure the boolean columns are genuinely Python `bool` dtype (`.astype(bool)`) rather than whatever pandas may have stored them as after the parquet round-trip -- downstream `.sum()` calls on these columns rely on them behaving like real booleans. It deliberately doesn't wrap

`pd.read_parquet` in a try/except -- the docstring explains why: letting the `FileNotFoundError` propagate up lets the caller (`main()`) decide how to present the error to the user, rather than this function making that decision.

Lines 57-105 -- `render_sidebar_filters()`

```
def render_sidebar_filters(df):
    """Draws every filter widget and returns the selections as a dict of
    column -> list of chosen values (empty list means "no filter applied")."""
    with st.sidebar:
        st.header("Filters")

        selections = {}

        if "violation_group" in df.columns:
            groups = sorted(df["violation_group"].dropna().unique())
            selections["violation_group"] = st.multiselect(
                "Violation Group", groups,
                default=["Speeding", "Registration / Plate", "License / Suspended"],
                key="filter_violation_group",
            )

        # ... same pattern repeated for state, vehicletype, gender, race, arrest_type ...

        st.button("Reset all filters", on_click=reset_filters)

    return selections
```

`with st.sidebar:` is a context manager -- every widget created inside this block renders in the left sidebar instead of the main page body. Each filter follows an identical pattern: check the column actually exists (defensive, in case a future data export is missing one), pull its distinct non-null values sorted alphabetically (`df[col].dropna().unique()`), and draw an `st.multiselect(...)` widget listing those as options. The `key=` argument on each widget is what makes `reset_filters()` (below) work at all -- Streamlit widgets without an explicit `key` are tracked by their position/label, but giving each one an explicit, stable key lets other code read or write that widget's value directly through `st.session_state["filter_..."]`. The `default=[...]` on the violation-group and state filters means the dashboard opens with a sensible non-empty view rather than showing every single record on first load; the other filters default to an empty list (no restriction). `st.button("Reset all filters", on_click=reset_filters)` wires the button to a callback function -- Streamlit runs `on_click` callbacks *before* the rest of the script reruns, which matters for the reset logic explained next. The function returns the `selections` dict so `main()` can pass it to `apply_filters()`.

Lines 108-115 -- `reset_filters()`

```
def reset_filters():
    """Callback for the reset button - clears every filter widget's stored
    state before Streamlit re-runs, so the multiselects actually go back to
    empty instead of just triggering a no-op rerun."""
    for key in FILTER_KEYS:
        widget_key = f"filter_{key}"
        if widget_key in st.session_state:
            st.session_state[widget_key] = []
```

This is the fix for a bug the original version of this dashboard had: a reset button that only called `st.rerun()` looks like it does something (the page does refresh) but doesn't actually clear any filter, because `st.rerun()` just re-executes the script -- it doesn't touch widget state. The real fix is this function: for each known filter column, it builds the matching widget key (`"filter_violation_group"`, `"filter_state"`, etc. -- matching exactly what `render_sidebar_filters()` used as each widget's `key=`) and, if that key currently exists in `st.session_state` (Streamlit's built-in dictionary-like store that persists widget values across reruns), overwrites it with an empty list.

Because this is registered as the button's `on_click` callback rather than being called inline in the main script body, Streamlit guarantees it runs *before* the multiselect widgets are re-drawn on the same rerun -- so by the time `render_sidebar_filters()` runs again and creates a widget with `key="filter_violation_group"`, Streamlit already has an empty list stored under that key and displays the widget as empty instead of keeping its old selection.

Lines 118-123 -- `apply_filters()`

```
def apply_filters(df, selections):
    filtered = df
    for column, chosen in selections.items():
        if chosen:
            filtered = filtered[filtered[column].isin(chosen)]
    return filtered
```

Starts with the full DataFrame and narrows it down one filter at a time. `selections.items()` walks each `(column_name, list_of_chosen_values)` pair from the dict `render_sidebar_filters()` built. `if chosen:` is the key line -- an empty list is falsy in Python, so a filter with nothing selected is simply skipped rather than being applied as "match nothing" (which is what `.isin([])` would otherwise do -- it would return an all-False mask and filter out every row). When a filter *does* have selections, `filtered[filtered[column].isin(chosen)]` keeps only rows whose value in that column is one of the chosen options. Doing this as a loop that reassigns `filtered` each time (rather than combining all conditions into one boolean expression at once) means each filter narrows down whatever the previous filter already narrowed -- functionally the same result either way for this kind of AND-combined filtering, but arguably easier to read one filter at a time.

Lines 126-141 -- `render_summary_metrics()`

```
def render_summary_metrics(filtered):
    st.subheader("Summary Statistics")
    cols = st.columns(4)
    cols[0].metric("Total Violations", f"{len(filtered):,}")
    cols[1].metric("Accident Contributed", f"{filtered['contributed_to_accident'].sum():,}")
    cols[2].metric("Seatbelt Violations", f"{filtered['belts'].sum():,}")
    cols[3].metric("Personal Injury Cases", f"{filtered['personal_injury'].sum():,}")

    if {"make", "model"}.issubset(filtered.columns) and len(filtered):
        combo = filtered.dropna(subset=["make", "model"]).groupby(["make", "model"]).size()
        if len(combo):
            top_make, top_model = combo.idxmax()
            st.caption(
                f"Most frequently cited vehicle in the current selection: "
                f"***{top_make} {top_model}** ({combo.max():,} stops)"
            )
```

`st.columns(4)` creates four equal-width side-by-side layout slots; each `cols[i].metric(label, value)` draws one of Streamlit's built-in "stat tile" widgets (a label, a big number, optionally a delta) into that slot. Every value is formatted with `,` for thousands separators so "12345" reads as "12,345". The block below the four metrics recomputes the same make+model grouping trick from `eda.py`'s `build_report()` -- but here it's computed on `filtered`, the currently-filtered subset, not the whole dataset, so this caption updates live as filters change (e.g. filtering down to only Maryland stops might surface a different top vehicle than the nationwide figure in `EDA_REPORT.md`). `{"make", "model"}.issubset(...)` checks both columns exist using set operations (equivalent to two separate `in` checks, just more compact for checking multiple names at once). `if len(filtered) / if len(combo)` guard against edge cases -- an empty filtered DataFrame, or one where every remaining row happens to be missing make or model -- where calling `.idxmax()` on an empty Series would raise an error.

Lines 144-169 -- `render_violation_charts()`

```

def render_violation_charts(filtered):
    with st.expander("Violation Categories & Patterns", expanded=True):
        col_left, col_right = st.columns(2)

        with col_left:
            st.subheader("Top Violation Groups")
            top_viol = filtered["violation_group"].value_counts().head(12).reset_index()
            top_viol.columns = ["Group", "Count"]
            fig = px.bar(
                top_viol, x="Count", y="Group", orientation="h",
                color="Count", color_continuous_scale=BLUE_SCALE,
                title="Most Common Violation Categories",
            )
            st.plotly_chart(fig, use_container_width=True)

        with col_right:
            # same pattern for top charge codes
            ...

```

`st.expander(title, expanded=True)` draws a collapsible section -- this one starts open, the ones in later functions start collapsed (`expanded=False`), so the dashboard's first screen isn't overwhelming but the most important section is visible immediately. Inside, `st.columns(2)` splits the expander into two side-by-side halves. On the left: `.value_counts().head(12)` counts and keeps the top 12 violation groups; `.reset_index()` turns the resulting Series (whose index is the group name and whose values are the counts) into a proper two-column DataFrame, and `top_viol.columns = ["Group", "Count"]` renames those two columns so Plotly's `x=/y=` arguments can refer to them by clear names. `px.bar(..., orientation="h", color="Count", color_continuous_scale=BLUE_SCALE)` draws a horizontal bar chart where each bar's own color also encodes its count value along the shared blue scale -- taller/darker bars are easy to spot at a glance in addition to reading the bar length itself. `st.plotly_chart(fig, use_container_width=True)` renders the figure and tells it to stretch to fill the available column width rather than a fixed pixel size. The charge-codes chart on the right follows the exact same five-step pattern, guarded by `if "charge" in filtered.columns:` in case that column were ever missing.

Lines 172-182 -- `render_demographics_charts()`

```

def render_demographics_charts(filtered):
    with st.expander("Demographics", expanded=False):
        st.subheader("Violations by Race and Gender")
        if {"race", "gender"}.issubset(filtered.columns):
            rg = filtered.groupby(["race", "gender"]).size().reset_index(name="Count")
            fig = px.bar(
                rg, x="race", y="Count", color="gender", barmode="group",
                color_discrete_sequence=CATEGORICAL_COLORS,
                title="Race x Gender Distribution", height=500,
            )
            st.plotly_chart(fig, use_container_width=True)

```

`filtered.groupby(["race", "gender"]).size()` counts how many rows fall into every combination of race and gender present in the filtered data; `.reset_index(name="Count")` turns that into a flat DataFrame with a `race` column, a `gender` column, and a `Count` column (the `name="Count"` argument names the count column directly, sparing a separate rename step like the one used for `top_viol` above). `px.bar(..., color="gender", barmode="group")` tells Plotly to draw one cluster of bars per race value, with a separate colored bar for each gender within each cluster (rather than `barmode="stack"`, which would stack them on top of each other instead). `color_discrete_sequence=CATEGORICAL_COLORS` -- note this is `color_discrete_sequence`, not `color_continuous_scale` like the earlier charts -- the difference matters: this chart's color represents a *category* (gender), so it needs Plotly's discrete/categorical coloring API, while the violation-count chart's color represented a *magnitude*, needing the continuous API instead.

Lines 185-211 -- `render_vehicle_charts()`

```
def render_vehicle_charts(filtered):
    with st.expander("Vehicle Information", expanded=False):
        col_left, col_right = st.columns(2)

        with col_left:
            st.subheader("Top Vehicle Makes")
            if "make" in filtered.columns:
                makes = filtered["make"].value_counts().head(15).reset_index()
                makes.columns = ["Make", "Count"]
                fig = px.treemap(
                    makes, path=["Make"], values="Count",
                    color="Count", color_continuous_scale=BLUE_SCALE,
                    title="Top Vehicle Makes Involved in Violations",
                )
                st.plotly_chart(fig, use_container_width=True)

        with col_right:
            # vehicle type breakdown, same value_counts + bar pattern as before
            ...
```

The left column uses `px.treemap` instead of a bar chart -- a treemap draws each category as a rectangle sized proportionally to its value, nested under a hierarchy given by `path=`. Here `path=["Make"]` is a single-level hierarchy (just makes, no nested sub-categories), so it's effectively a "proportional area" alternative to a bar chart -- useful when there are many categories (up to 15 here) and relative size matters more than precise ranking. `values="Count"` tells Plotly which column determines each rectangle's area; `color="Count"` with the same `BLUE_SCALE` again ties the shading to magnitude, consistent with the other charts. The right column's vehicle-type breakdown reuses the same "value_counts -> rename columns -> `px.bar(orientation="h")`" pattern seen earlier, but with a solid single color (`color_discrete_sequence=[CATEGORICAL_COLORS[0]]`) instead of a continuous scale, since this bar chart isn't meant to double-encode magnitude through color the way the treemap does.

Lines 214-225 -- `render_enforcement_charts()`

```
def render_enforcement_charts(filtered):
    with st.expander("Enforcement & Arrest", expanded=False):
        st.subheader("Arrest Type Distribution")
        if "arrest_type" in filtered.columns:
            arrest_counts = filtered["arrest_type"].value_counts().reset_index()
            arrest_counts.columns = ["Arrest Type", "Count"]
            fig = px.bar(
                arrest_counts, x="Count", y="Arrest Type", orientation="h",
                color_discrete_sequence=[CATEGORICAL_COLORS[1]],
                title="Citations, Warnings & Other Outcomes",
            )
            st.plotly_chart(fig, use_container_width=True)
```

Same pattern once more, this time with no `.head(n)` cutoff -- arrest type only has around 15 distinct values in the data (see README), so showing all of them doesn't clutter the chart the way showing every single vehicle make would. Uses `CATEGORICAL_COLORS[1]` (the orange slot) specifically so this chart doesn't look identical to the blue vehicle-type chart right above it in the page flow.

Lines 228-235 -- `render_download_button()`

```
def render_download_button(filtered):
    csv_bytes = filtered.to_csv(index=False).encode("utf-8")
    st.download_button(
        "Download Filtered Data (CSV)",
```

```

        data=csv_bytes,
        file_name=f"filtered_traffic_violations_{len(filtered)}rows.csv",
        mime="text/csv",
    )

```

`filtered.to_csv(index=False)` renders the currently filtered DataFrame as CSV text (again skipping the meaningless row-index column); `.encode("utf-8")` turns that text into raw bytes, which is what `st.download_button` expects for its `data=` argument. The generated filename embeds the row count (`f"...{len(filtered)}rows.csv"`) so a user who downloads the same filter a few times, or compares two different filter selections, gets distinguishable filenames rather than every download being named the exact same thing.

Lines 238-270 -- `main()`

```

def main():
    configure_page()

    with st.spinner("Loading data..."):
        try:
            df = load_data()
        except FileNotFoundError:
            st.error(f"Could not find {DATA_FILE}. Run `python clean_data.py` first.")
            st.stop()

    st.title("Traffic Violations Insight Dashboard")
    st.markdown(
        f"""Interactive analysis of traffic violations** \n
        Showing **{len(df):,}** processed records - Educational project \n
        Data source: raw_traffic.csv (cleaned & enriched)"""
    )

    selections = render_sidebar_filters(df)
    filtered = apply_filters(df, selections)

    if filtered.empty:
        st.warning("No records match the current filters. Try clearing some selections.")
        st.stop()

    render_summary_metrics(filtered)
    render_download_button(filtered)
    render_violation_charts(filtered)
    render_demographics_charts(filtered)
    render_vehicle_charts(filtered)
    render_enforcement_charts(filtered)

    st.markdown("---")
    st.caption("Data processed from raw_traffic.csv - Educational project - Built with Streamlit & Plotly")

```

The top-level orchestrator -- this is the function that actually runs top-to-bottom on every Streamlit rerun.

`configure_page()` must go first (see that function's notes above). `with st.spinner("Loading data..."):` shows a small loading spinner in the UI for the duration of the indented block -- in practice this only matters on the very first run, since `load_data()` is cached afterward and returns near-instantly on later reruns. The `try/except FileNotFoundError` around `load_data()` is where that function's *deliberate* choice not to catch its own error (see `load_data()`'s notes) pays off: `main()` catches it here instead, shows a friendly `st.error(...)` message with the exact fix ("run `python clean_data.py` first"), and calls `st.stop()` -- which halts execution of the rest of the script immediately, so nothing below tries to use a `df` that was never successfully created. After a successful load, it prints the title and a short intro line using an f-string with embedded `\n` characters (in Streamlit markdown, a trailing double-space or, as here, an explicit `\n` inside a markdown string, forces a line break within one `st.markdown` call rather than starting a new paragraph). Then: `render_sidebar_filters(df)` draws all the filter widgets and returns the current selections; `apply_filters(df, selections)` narrows the full dataset down to just the matching rows.

`if filtered.empty: ... st.stop()` is a safety valve -- if someone picks a combination of filters that matches zero rows, this shows a warning and stops, rather than letting every chart function below try to build a chart from an empty DataFrame (some Plotly chart types raise ugly errors on genuinely empty input). Assuming rows remain, every render function runs in sequence, each one drawing its own section of the page using the same `filtered` DataFrame -- this is the "pass filtered data down to each independent render function" pattern that keeps each chart function simple and testable in isolation, since none of them need to know anything about *how* the filtering happened, only that they were handed the DataFrame to draw from.

Lines 273-274 -- entry point

```
if __name__ == "__main__":  
    main()
```

Same idiom as the other two scripts. In practice, Streamlit's own `streamlit run app.py` command handles executing this file in a way that always satisfies `__name__ == "__main__"`, so this line is really just standard practice rather than something Streamlit specifically requires.